>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 6

Loops: the for Loop



Counting, accumulating, nested loops, break and continue

Asst. Prof. Hasan Oğuz, PhD

Istanbul Okan University · Vocational School

2026–2027 Fall Semester

>_ LEARNING OBJECTIVES

By the End of This Week You Can …

Explain the for loop

Name its four parts (initialization, condition, update, body) and their execution order.

Draw and trace

Draw the flowchart of a for loop and fill in a trace table by hand.

Accumulate

Compute sums, products, counts, averages, minimum and maximum with one loop.

Nest loops

Print grids and star patterns; predict how many times a body runs.

break and continue

Leave a loop early or skip one pass; test primes efficiently.

Avoid classic bugs

Off-by-one, stray semicolon, changing i, infinite loops, overflow.

From Deciding Once to Repeating Many Times

Sequence

Weeks 3–4 · one step after another

Selection

Week 5 · if / else / switch

Repetition

Weeks 6–7 · for / while / do-while

Week 1: sum 1..N with WHILE

```
START
  READ N
  sum ← 0
  i ← 1
  WHILE i ≤ N
    sum ← sum + i
    i ← i + 1
  END WHILE
  PRINT sum
END
```

Same algorithm with FOR

```
START
  READ N
  sum ← 0
  FOR i ← 1 TO N
    sum ← sum + i
  END FOR
  PRINT sum
END
```

The FOR line packs the three counter jobs into one line: **start** (i ← 1), **test** (i ≤ N), **step** (i ← i + 1). C#'s for header does exactly the same.



PART 1

Why Loops? Anatomy of the for Loop

Four parts, one fixed execution order, one flowchart

Printing 1 to 100: Without and With a Loop

Without a loop: 100 lines

```
Console.WriteLine(1);
Console.WriteLine(2);
Console.WriteLine(3);
// ... 96 more almost identical lines ...
Console.WriteLine(100);
```

With a for loop: 4 lines

```
for (int i = 1; i <= 100; i++)
{
    Console.WriteLine(i);
}
```

100 → 4 lines

The loop body is written once and executed 100 times.

Size at run time

Replace 100 by n and the user decides how many lines are printed.

Fewer places for bugs

One line to check instead of a hundred nearly identical ones.

>_ ANATOMY

The Four Parts of a for Loop

```
for (int i = 1; i <= 5; i++)
     ①        ②      ④
{
    Console.Write($"{i} ");
}
```

③

Console

```
1 2 3 4 5
```

① Initialization `int i = 1`

Runs once, before anything else. Declares the loop variable.

② Condition `i <= 5`

Tested before every pass: true → run the body, false → leave the loop.

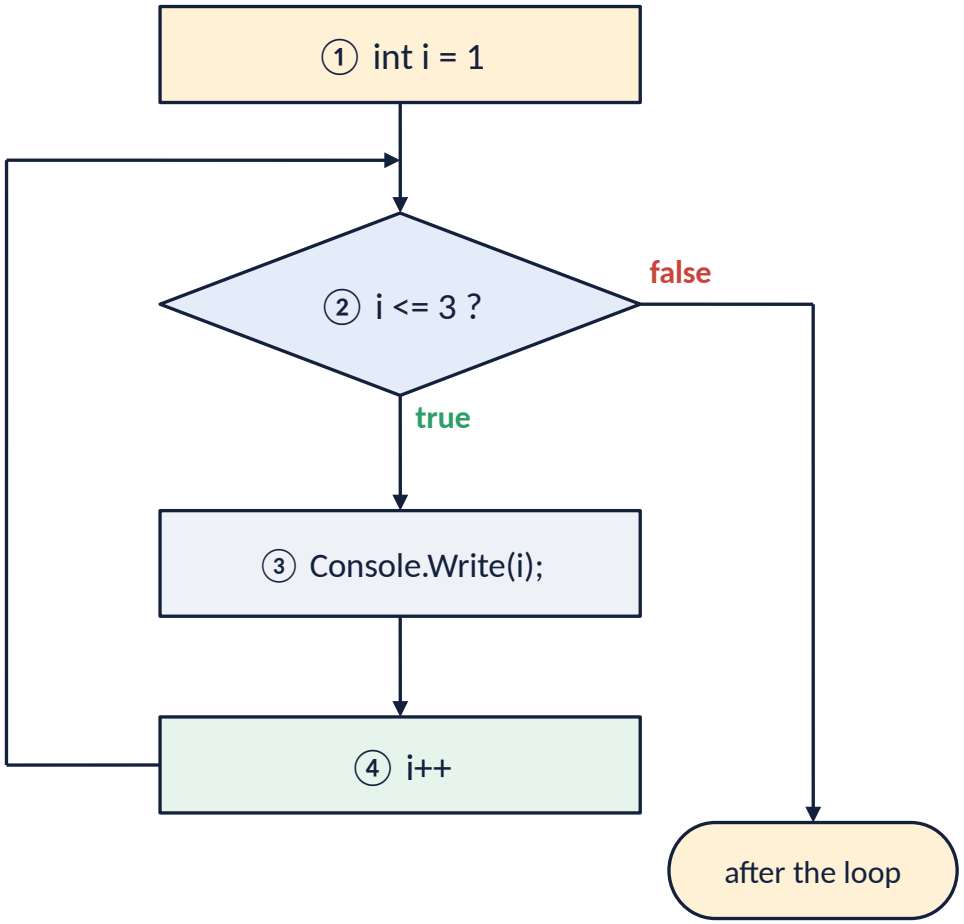
③ Body `{ ... }`

The statements that are repeated.

④ Update `i++`

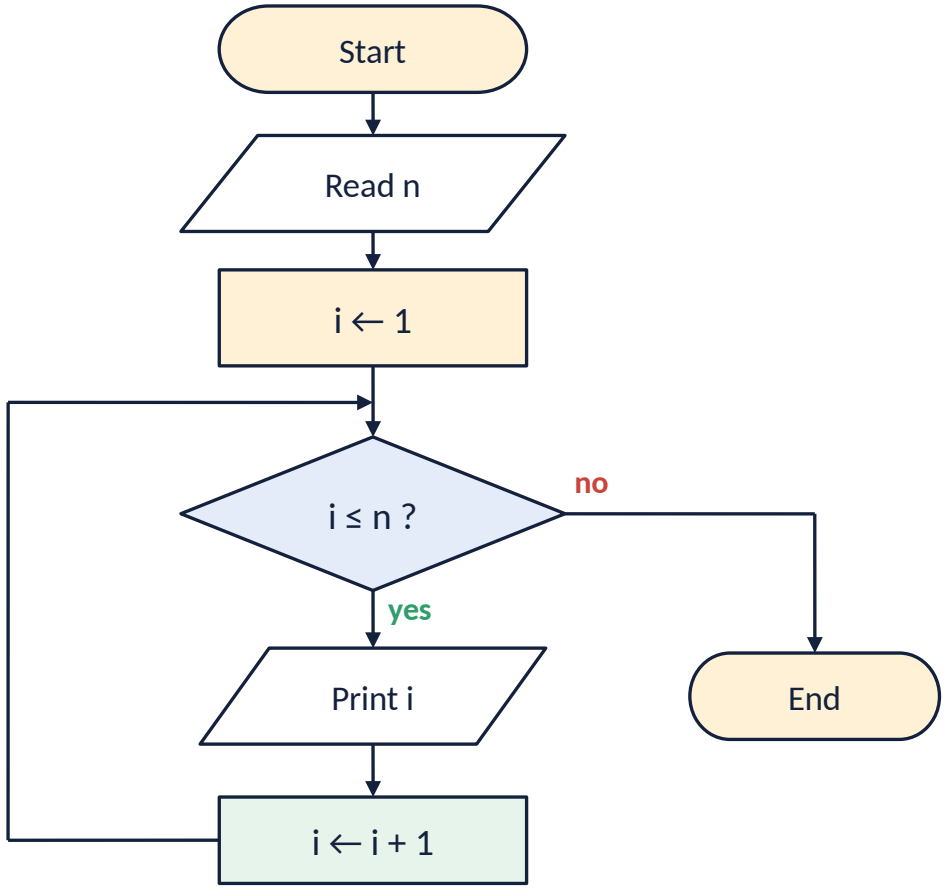
Runs after every pass, then back to ②.

Execution Order: ① → ② → ③ → ④ → ② → …



#	Part	What happens	Output
1	① init	`i = 1`	
2	② test	`1 <= 3 → true`	
3	③ body	`Write(1)`	`1`
4	④ update	`i++ → i = 2`	
5	② test	`2 <= 3 → true`	
6	③ body	`Write(2)`	`12`
7	④ update	`i++ → i = 3`	
8	② test	`3 <= 3 → true`	
9	③ body	`Write(3)`	`123`
10	④ update	`i++ → i = 4`	
11	**② test**	**`4 <= 3 → false: exit`**	**`123`**

The for Loop as a Flowchart, in Pseudocode and in C#



Pseudocode

```
START
  READ n
  FOR i ← 1 TO n
    PRINT i
  END FOR
END
```

C#

```
int n = int.Parse(Console.ReadLine());
for (int i = 1; i <= n; i++)
{
    Console.WriteLine(i);
}
```

Tracing a for Loop by Hand: 1 + 2 + … + 5

Program.cs (excerpt)

```
int sum = 0;
for (int i = 1; i <= 5; i++)
{
    sum += i;
}
Console.WriteLine(sum);
```

Console

```
15
```

Condition tested 6 times, body run 5 times.

Pass	i (before test)	i <= 5 ?	sum (after body)
start	1	–	0
1	1	true	1
2	2	true	3
3	3	true	6
4	4	true	10
5	5	true	15
exit	**6**	**false**	**15**

Check: 1 + 2 + 3 + 4 + 5 = 5 · 6 / 2 = 15 (Gauss, Week 1)



PART 2

Counting and Accumulating

Up, down, in steps · sums, products, counters · min and max

Counting Up, Down and in Steps

Pattern	for header	Output for n = 10
Up	`for (int i = 1; i <= n; i++)`	`1 2 3 4 5 6 7 8 9 10`
Down	`for (int i = n; i >= 1; i--)`	`10 9 8 7 6 5 4 3 2 1`
Even	`for (int i = 0; i <= n; i += 2)`	`0 2 4 6 8 10`
By −5	`for (int i = 5 * n; i > 0; i -= 5)`	`50 45 40 35 30 25 20 15 10 5`

Console

```
Count up to: 10
Up:     1 2 3 4 5 6 7 8 9 10
Down:   10 9 8 7 6 5 4 3 2 1
Even:   0 2 4 6 8 10
By -5:  50 45 40 35 30 25 20 15 10 5
```



Direction rule

Up: start low, test < or <=, update ++ or +=.
Down: start high, test > or >=, update -- or -=.

How Many Times Does the Body Run?

for header	values of i	passes
`for (int i = 1; i <= 10; i++)`	1, 2, …, 10	10
`for (int i = 0; i < 10; i++)`	0, 1, …, 9	10
`for (int i = 1; i < 10; i++)`	1, 2, …, 9	9
`for (int i = 0; i <= 20; i += 2)`	0, 2, …, 20	11
`for (int i = 10; i > 0; i -= 3)`	10, 7, 4, 1	4
`for (int i = 1; i <= 100; i *= 2)`	1, 2, 4, …, 64	7
`for (int i = 5; i < 5; i++)`	(none)	0

COUNTING RULE

`i = a; i <= b; i += s`

passes = (b − a) / s + 1

integer division, when a ≤ b



Nested loops

Outer n passes, inner m passes → n · m body runs.
Inner j ≤ i → 1 + 2 + … + n = n(n + 1) / 2.

The Loop Variable Lives Only Inside the Loop

Does not compile

```
int sum = 0;
for (int i = 1; i <= 5; i++)
{
    sum += i;
}
Console.WriteLine($"Last i = {i}");
```

error CS0103: The name 'i' does not exist in the current context

Fix: declare i before the loop

```
int sum = 0;
int i;                  // declared outside
for (i = 1; i <= 5; i++)
{
    sum += i;
}
Console.WriteLine($"Last i = {i}");
```

Console

```
Last i = 6
```



Two loops in a row may both declare int i: each i is a new variable that disappears at the loop's closing brace.

Accumulators and Counters

Sum

```
before loop: int sum = 0;
inside loop: sum += x;
```

Start at 0: adding 0 changes nothing.

Product

```
before loop: long product = 1;
inside loop: product *= x;
```

Start at 1: starting at 0 keeps the result 0.

Counter

```
before loop: int count = 0;
inside loop: if (x >= 50) count++;
```

Counts the passes that meet a condition.

Average

```
after loop:  if (count > 0)
                 avg = (double)sum / count;
```

After the loop, never inside; guard count > 0.

Sum, Average and Pass Count of N Grades

Problem

Read how many grades there are, then each grade. Print the sum, the average and how many grades are at least 50.

Test input	Expected output
5 grades: 72.5, 45, 88, 60, 39.5	sum 305.00, average 61.00, passed 3 of 5
0 grades	No grades entered.

Pseudocode

```
START
  READ count
  sum ← 0
  passed ← 0
  FOR i ← 1 TO count
    READ grade
    sum ← sum + grade
    IF grade ≥ 50 THEN
      passed ← passed + 1
    END IF
  END FOR
  IF count > 0 THEN
    PRINT sum, sum / count, passed
  ELSE
    PRINT "No grades entered."
  END IF
END
```

Sum, Average and Pass Count: C# and a Test Run

06_02_SumAndAverage/Program.cs (excerpt)

```
Console.Write("How many grades? ");
int count = int.Parse(Console.ReadLine());

double sum = 0;   // accumulator: starts at 0
int passed = 0;   // counter: starts at 0

for (int i = 1; i <= count; i++)
{
    Console.Write($"Grade {i}: ");
    double grade = double.Parse(Console.ReadLine());
    sum += grade;              // add to the running total
    if (grade >= 50)
    {
        passed++;              // count the passing grades
    }
}
```

Console

```
How many grades? 5
Grade 1: 72.5
Grade 2: 45
Grade 3: 88
Grade 4: 60
Grade 5: 39.5
Sum     = 305.00
Average = 61.00
Passed  = 3 of 5
```

After the loop

average = sum / count, printed only if count > 0.

Factorial n! with a Product Accumulator

06_03_Factorial/Program.cs

```
Console.Write("n = ");
int n = int.Parse(Console.ReadLine());

long factorial = 1;   // product: starts at 1, not 0
for (int i = 1; i <= n; i++)
{
    factorial *= i;   // factorial = factorial * i
    Console.WriteLine($"{i,2}! = {factorial,26:N0}");
}
Console.WriteLine($"long.MaxValue = {long.MaxValue:N0}");
```

Console: n = 22 (last 9 lines)

```
15! =           1,307,674,368,000
16! =          20,922,789,888,000
17! =         355,687,428,096,000
18! =       6,402,373,705,728,000
19! =     121,645,100,408,832,000
20! =   2,432,902,008,176,640,000
21! = -4,249,290,049,419,214,848
22! = -1,250,660,718,674,968,576
long.MaxValue = 9,223,372,036,854,775,807
```

Silent overflow after 20!

21! ≈ $5.1 \cdot 10^{19}$ > long.MaxValue ≈ $9.2 \cdot 10^{18}$. The value wraps around and turns negative; no error is reported.

Detect it

factorial = checked(factorial * i);
throws an OverflowException instead.

Using the Loop Variable in a Calculation

06_04_MultiplicationTable/Program.cs

```
Console.Write("Table of: ");
int number = int.Parse(Console.ReadLine());

for (int i = 1; i <= 10; i++)
{
    Console.WriteLine($"{number} x {i,2} = {number * i,3}");
}
```

Console

```
Table of: 7
7 x  1 =   7
7 x  2 =  14
7 x  3 =  21
7 x  4 =  28
7 x  5 =  35
7 x  6 =  42
7 x  7 =  49
7 x  8 =  56
7 x  9 =  63
7 x 10 =  70
```



i is readable data

Use i in expressions and output: number * i, $"{i,2}". Change it only in the update part of the header.



All tables 1–10 at once? Put this loop inside another loop (Part 3).

Compound Interest, Year by Year

06_13_CompoundInterest/Program.cs

```
Console.Write("Deposit (TL): ");
decimal balance = decimal.Parse(Console.ReadLine());
Console.Write("Annual interest rate (%): ");
decimal rate = decimal.Parse(Console.ReadLine());
Console.Write("Years: ");
int years = int.Parse(Console.ReadLine());

Console.WriteLine("Year      Interest       Balance");
for (int year = 1; year <= years; year++)
{
    decimal interest = balance * rate / 100;  // this year
    balance += interest;                      // add to balance
    Console.WriteLine($"{year,4} {interest,13:F2} {balance,13:F2}");
}
```



decimal for money (Week 3). **`rate / 100`** turns 40 into 0.40.

Console

```
Deposit (TL): 10000
Annual interest rate (%): 40
Years: 5
Year      Interest       Balance
   1       4000.00      14000.00
   2       5600.00      19600.00
   3       7840.00      27440.00
   4      10976.00      38416.00
   5      15366.40      53782.40
```

CHECK WITH THE FORMULA

$$A = P \cdot (1 + r)^n$$

$10\,000 \cdot 1.40^5 = 53\,782.40$ ✓

Smallest and Largest of N Numbers: No Arrays Needed

06_12_MinMax/Program.cs (excerpt)

```
double min = 0;
double max = 0;
for (int i = 1; i <= count; i++)
{
    Console.Write($"Number {i}: ");
    double x = double.Parse(Console.ReadLine());
    if (i == 1 || x < min)     // first number, or a new minimum
    {
        min = x;
    }
    if (i == 1 || x > max)     // first number, or a new maximum
    {
        max = x;
    }
}
```

Console

```
How many numbers? 5
Number 1: 17
Number 2: -4
Number 3: 23.5
Number 4: 8
Number 5: 0
Smallest = -4
Largest  = 23.5
Range    = 27.5
```

The first number is both the smallest and the largest so far.

Why not min = 0?

For 17, 23.5, 8 the "minimum" would stay 0, a value nobody entered.

Fibonacci: Carrying Two Values to the Next Pass

06_10_Fibonacci/Program.cs

```
Console.Write("How many terms? ");
int n = int.Parse(Console.ReadLine());

long previous = 0;   // F(0)
long current = 1;    // F(1)
for (int i = 1; i <= n; i++)
{
    Console.Write($"{previous} ");
    long next = previous + current;  // sum of the last two
    previous = current;              // shift one step forward
    current = next;
}
Console.WriteLine();
```

Console

```
How many terms? 12
0 1 1 2 3 5 8 13 21 34 55 89
```

i	printed	next	previous	current
start			0	1
1	0	1	1	1
2	1	2	1	2
3	1	3	2	3
4	2	5	3	5
5	3	8	5	8
6	5	13	8	13



n = 6 prints: **0 1 1 2 3 5**



PART 3

Nested Loops

A loop inside a loop: grids, tables and star patterns

How Nested Loops Run

Program.cs (excerpt)

```
for (int i = 1; i <= 3; i++)          // outer
{
    for (int j = 1; j <= 2; j++)      // inner
    {
        Console.Write($"({i},{j}) ");
    }
    Console.WriteLine();
}
```

Console

```
(1,1) (1,2)
(2,1) (2,2)
(3,1) (3,2)
```

Like a clock

The inner loop makes a full turn for every single step of the outer loop.

Inner loop restarts

Its initialization j = 1 runs again at every outer pass.

3 × 2 = 6 passes

Outer passes × inner passes = total runs of the inner body.

Multiplication Grid: Rows × Columns

06_05_MultiplicationGrid/Program.cs

```
Console.Write("Grid size: ");
int size = int.Parse(Console.ReadLine());

for (int row = 1; row <= size; row++)      // outer: rows
{
    for (int col = 1; col <= size; col++)  // inner: columns
    {
        Console.Write($"{row * col,4}");   // 4 chars wide
    }
    Console.WriteLine();                   // end of row
}
```

Console

```
Grid size: 10
   1   2   3   4   5   6   7   8   9  10
   2   4   6   8  10  12  14  16  18  20
   3   6   9  12  15  18  21  24  27  30
   4   8  12  16  20  24  28  32  36  40
   5  10  15  20  25  30  35  40  45  50
   6  12  18  24  30  36  42  48  54  60
   7  14  21  28  35  42  49  56  63  70
   8  16  24  32  40  48  56  64  72  80
   9  18  27  36  45  54  63  72  81  90
  10  20  30  40  50  60  70  80  90 100
```



Where does WriteLine() go?

After the inner loop, inside the outer loop: one row per line.



10 × 10 = 100

products written by the inner body.

Star Patterns: Rectangle and Right Triangle

```
// A) rectangle: every row has the same number of stars
Console.WriteLine("A) Rectangle");
for (int r = 1; r <= rows; r++)
{
    for (int c = 1; c <= 2 * rows; c++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}

// B) right triangle: row r has r stars
Console.WriteLine("B) Right triangle");
for (int r = 1; r <= rows; r++)
{
    for (int c = 1; c <= r; c++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}
```

Console

```
Rows: 4
A) Rectangle
********
********
********
********
B) Right triangle
*
**
***
****
C) Pyramid
   *
  ***
 *****
*******
```

Pyramid: Two Inner Loops per Row

row r	spaces = rows − r	stars = 2r − 1	line
1	3	1	`...*`
2	2	3	`..***`
3	1	5	`.*****`
4	0	7	`*******`

Console (part C, rows = 4)

```
C) Pyramid
   *
  ***
 *****
*******
```

```
// C) pyramid: row r has rows - r spaces, 2r - 1 stars
Console.WriteLine("C) Pyramid");
for (int r = 1; r <= rows; r++)
{
    for (int s = 1; s <= rows - r; s++)
    {
        Console.Write(" ");
    }
    for (int c = 1; c <= 2 * r - 1; c++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}
```

Predict the Output: Nested Loops

Q1

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= i; j++)
    {
        Console.Write(j);
    }
    Console.WriteLine();
}
```

Q2

```
for (int i = 3; i >= 1; i--)
{
    for (int j = 1; j <= i; j++)
    {
        Console.Write("#");
    }
    Console.WriteLine();
}
```

Q3

```
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= 4; j++)
    {
        if ((i + j) % 2 == 0)
        {
            Console.Write("#");
        }
        else
        {
            Console.Write(".");
        }
    }
    Console.WriteLine();
}
```



Tip: make a small table with columns i, j and "printed" before you write the output.

Answers: Nested Loops

Q1

```
1
12
123
```

Q2

```
###
##
#
```

Q3

```
#.#.
.#.#
#.#.
.#.#
```

Inner bound = i

Row i prints 1 … i: a triangle of numbers.

Outer counts down

i = 3, 2, 1: the rows get shorter.

Parity of i + j

Even → #, odd → . gives a 4 × 4 checkerboard.



PART 4

break, continue and Classic Algorithms

Leaving early, skipping a pass, primes, FizzBuzz, and the classic mistakes

break Leaves the Loop, continue Skips One Pass

06_07_BreakContinue/Program.cs (part 2)

```
int sum = 0;
int used = 0;
for (int k = 1; k <= 10; k++)
{
    Console.Write($"Number {k} (0 = stop): ");
    int x = int.Parse(Console.ReadLine());
    if (x == 0)
    {
        break;                  // leave the loop immediately
    }
    if (x < 0)
    {
        Console.WriteLine("  negative -> skipped");
        continue;                // go on with the next k
    }
    sum += x;
    used++;
}
Console.WriteLine($"Sum of {used} positive numbers = {sum}");
```



break
Leaves the loop immediately.



continue
Skips to the update, then the next test.

Console

```
1 2 4 5 7 8 10 11 13 14
Number 1 (0 = stop): 8
Number 2 (0 = stop): -3
  negative -> skipped
Number 3 (0 = stop): 12
Number 4 (0 = stop): 5
Number 5 (0 = stop): 0
Sum of 3 positive numbers = 25
```

Prime Test: Search for a Divisor, Stop at the First

Why only up to √n?

Divisor pairs of 36:
1·36 2·18 3·12 4·9 6·6
One partner of every pair is ≤ √36 = 6. For n = 1 000 003: 999 divisions instead of about a million.

Console

```
n = 97
97 is prime
Divisions tried: 8
```

Console

```
n = 91
91 = 7 x 13
91 is not prime
Divisions tried: 6
```

06_08_PrimeTest/Program.cs (excerpt)

```
bool isPrime = n >= 2;          // 0, 1, negatives: not prime
int limit = (int)Math.Sqrt(n);  // largest divisor to test
int checks = 0;                 // divisions tried

for (int d = 2; d <= limit; d++)
{
    checks++;
    if (n % d == 0)
    {
        Console.WriteLine($"{n} = {d} x {n / d}");
        isPrime = false;
        break;                  // one divisor is enough
    }
}
```

All Primes up to N: a Search Loop Inside a Counting Loop

```
for (int n = 2; n <= max; n++)          // outer: candidates
{
    bool isPrime = true;
    for (int d = 2; d * d <= n; d++)    // inner: divisors
    {
        if (n % d == 0)
        {
            isPrime = false;
            break;                      // inner loop only
        }
    }
    if (isPrime)
    {
        found++;
        Console.Write($"{n,4}");
        if (found % 10 == 0)
        {
            Console.WriteLine();        // 10 per line
        }
    }
}
```

Console

```
Primes up to: 100
   2   3   5   7  11  13  17  19  23  29
  31  37  41  43  47  53  59  61  67  71
  73  79  83  89  97
25 primes found.
```

break is local

It leaves only the innermost loop.

25 primes below 100

Inner loop tries at most 8 divisors.

FizzBuzz: a Loop with an else-if Ladder

```
for (int i = 1; i <= n; i++)
{
    if (i % 15 == 0)        // divisible by 3 AND 5: test first
    {
        Console.WriteLine("FizzBuzz");
    }
    else if (i % 3 == 0)
    {
        Console.WriteLine("Fizz");
    }
    else if (i % 5 == 0)
    {
        Console.WriteLine("Buzz");
    }
    else
    {
        Console.WriteLine(i);
    }
}
```

Console

```
Up to: 15
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

Why test 15 first?

If i % 3 came first, 15 would print Fizz: the FizzBuzz branch would never run.

Off-by-One and the Stray Semicolon

Off-by-one: < instead of <=

```
// Goal: print 1 to 10
for (int i = 1; i < 10; i++)
{
    Console.Write($"{i} ");
}
```

Console

```
1 2 3 4 5 6 7 8 9
```

Only 9 numbers. Use i <= 10. Always check the first and the last value of i.

Semicolon after the header

```
int count = 0;
for (int i = 1; i <= 5; i++);
{
    count++;
}
Console.WriteLine(count);
```

Build + Console

```
warning CS0642: Possible mistaken empty statement
1
```

The loop body is the empty statement ";". The block runs once: count = 1, not 5.

Changing i, Endless Loops and Misplaced Accumulators

Changing i inside the body

```
for (int i = 1; i <= 10; i++)
{
    Console.Write($"{i} ");
    i++;   // second update!
}
```

Prints 1 3 5 7 9. Let the header do the updating.

Wrong direction, no condition

```
for (int i = 1; i <= 10; i--) { ... }
for (;;) { ... }
```

Line 1: 2 147 483 650 passes (int overflow). Line 2: endless, Ctrl+C.

double as a counter

```
for (double x = 0; x != 1.0; x += 0.1)
{
    ...
}
```

x reaches 0.9999999999999999, never exactly 1.0: endless. Count with int.

Accumulator inside the loop

```
for (int i = 1; i <= 3; i++)
{
    int sum = 0;   // reset every pass!
    sum += i;
}
```

sum restarts at 0 and is gone after the loop. Declare it before the loop.

Predict the Output: Counting, break, continue

Q4

```
int count = 0;
for (int i = 0; i < 4; i++)
{
    for (int j = i; j < 4; j++)
    {
        count++;
    }
}
Console.WriteLine(count);
```

Q5

```
for (int i = 1; i <= 10; i++)
{
    if (i * i > 20)
    {
        break;
    }
    Console.Write(i);
}
```

Q6

```
for (int i = 1; i <= 3; i++)
{
    for (int j = 1; j <= 3; j++)
    {
        if (i == j)
        {
            continue;
        }
        Console.Write($"{i}{j} ");
    }
}
```



Q4: count the inner passes for i = 0, 1, 2, 3. Q5: check i * i before printing. Q6: which pairs are skipped?

Answers: Counting, break, continue

Q4

```
10
```

Q5

```
1234
```

Q6

```
12 13 21 23 31 32
```

Triangular count

j runs from i to 3:
4 + 3 + 2 + 1 = 10
= 4 · 5 / 2

break at i = 5

i = 1..4: i * i = 1, 4, 9, 16 ≤ 20, so i is printed; at i = 5, 25 > 20 → break before Write.

continue skips i == j

3 × 3 = 9 pairs minus 11, 22, 33 → 6 pairs.
The inner update j++ still runs after continue.

>_ IN-CLASS EXERCISES · PAIRS

In-Class Exercises

A. Odd sum ★

Read N; print 1 + 3 + 5 + … up to N. Use a step of 2. (N = 10 → 25)

B. Powers of 2 ★

Read N; print 2^0, 2^1, …, 2^N with a product accumulator (no Math.Pow).

C. Divisors ★★

Read n; list its divisors, count them, and say whether n is perfect (28 is).

D. Hollow square ★★

Read n; print an n × n square of * that is empty inside.

E. Sign counter ★★

Read N integers; count the positive, negative and zero values.

F. Floyd's triangle ★★

Rows 1 / 2 3 / 4 5 6 / …: a counter that keeps running across rows.

>_ HOMEWORK

Homework for Week 7

#	Task	Level
H1	Sum of squares $1^2 + \ldots + n^2$ with a loop; compare with $n(n + 1)(2n + 1) / 6$	★
H2	Celsius → Fahrenheit table from −20 °C to 40 °C in steps of 10	★
H3	Diamond of stars: a pyramid of n rows plus the same pyramid upside down	★★
H4	Read N exam scores; print average, highest, lowest and number of passes	★★
H5	All perfect numbers up to 10 000 (nested loops)	★★★
H6	On paper: trace table of the loop given in the notes (continue + accumulator)	★

Method

Pseudocode or flowchart → C# → test table.

Questions?

Ask in the course group.

>_ SUMMARY

Key Takeaways

01 for (init; condition; update) runs init once, then test → body → update until the test is false.

02 Draw it as a flowchart with a back-edge; trace it with one row per pass plus the exit row.

03 Accumulators start at 0 (sum, count) or 1 (product) before the loop; averages come after it.

04 Nested loops: the inner loop runs completely for every outer pass (n · m body runs).

05 break leaves the (innermost) loop; continue jumps to the update.

06 Check the first and last value of i; no ; after the header; never change i in the body.

NEXT WEEK

Loops: while and do-while



MCMP113
Course materials